

User Input

Generated from .github/agents/speckit.implement.agent.md

Artifact type: other

Table of contents

User Input

Pre-Execution Checks

Outline

Mandatory Post-Execution Hooks

Completion Report

Done When

Body

User Input

``text \$ARGUMENTS ``` You **MUST** consider the user input before proceeding (if not empty).

Pre-Execution Checks

Check for extension hooks (before implementation): - Check if `.specify/extensions.yml` exists in the project root. - If it exists, read it and look for entries under the `hooks.before_implement` key - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally - Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook `condition` expressions: - If the hook has no `condition` field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty `condition`, skip the hook and leave condition evaluation to the HookExecutor implementation - For each executable hook, output the following based on its `optional` flag: - **Optional hook** (`optional: true`): `` ## Extension Hooks **Optional Pre-Hook**: {extension} Command: `{command}` Description: {description} Prompt: {prompt} To execute: `{command}` ``` - **Mandatory hook** (`optional: false`): `` ## Extension Hooks **Automatic Pre-Hook**: {extension} Executing: `{command}` EXECUTE_COMMAND: {command} Wait for the result of the hook command before proceeding to the Outline. ``` After emitting the block above you **MUST** actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation

may differ from the literal `{command}` id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or $speckit-...). Emitting the block alone does not run the hook. - If no hooks are registered or .specify/extensions.yml` does not exist, skip silently`

Outline

1. Run `.specify/scripts/powershell/check-prerequisites.ps1 -Json -RequireTasks`
-IncludeTasks` from repo root and parse FEATURE_DIR and AVAILABLE_DOCS list. All paths must be absolute. For single quotes in args like "I'm Groot", use escape syntax: e.g. 'I\'m Groot' (or double-quote if possible: "I'm Groot"). 2. **Check checklists status** (if FEATURE_DIR/checklists/ exists): - Scan all checklist files in the checklists/ directory - For each checklist, count: - Total items: All lines matching ``- []` or `- [X]` or `- [x]` - Completed items: Lines matching `- [X]` or `- [x]` - Incomplete items: Lines matching `- []` - Create a status table: ```text | Checklist | Total | Completed | Incomplete | Status |
|-----|-----|-----|-----|-----| | ux.md | 12 | 12 | 0 | ✓ PASS | | test.md | 8 | 5 | 3 | ✗ FAIL | | security.md | 6 | 6 | 0 | ✓ PASS | ``` - Calculate overall status: - PASS: All checklists have 0 incomplete items - FAIL: One or more checklists have incomplete items - If any checklist is incomplete: - Display the table with incomplete item counts - STOP and ask: "Some checklists are incomplete. Do you want to proceed with implementation anyway? (yes/no)" - Wait for user response before continuing - If user says "no" or "wait" or "stop", halt execution - If user says "yes" or "proceed" or "continue", proceed to step 3 - If all checklists are complete: - Display the table showing all checklists passed - Automatically proceed to step 3 3. Load and analyze the implementation context: - REQUIRED: Read tasks.md for the complete task list and execution plan - REQUIRED: Read plan.md for tech stack, architecture, and file structure - IF EXISTS: Read data-model.md for entities and relationships - IF EXISTS: Read contracts/ for API specifications and test requirements - IF EXISTS: Read research.md for technical decisions and constraints - IF EXISTS: Read .specify/memory/constitution.md for governance constraints - IF EXISTS: Read quickstart.md for integration scenarios 4. Project Setup Verification: - REQUIRED: Create/verify ignore files based on actual project setup: Detection & Creation Logic: - Check if the following command succeeds to determine if the repository is a git repo (create/verify .gitignore if so): ```sh git rev-parse --git-dir 2>/dev/null ``` - Check if Dockerfile* exists or Docker in plan.md → create/verify .dockerignore - Check if .eslintrc* exists → create/verify .eslintignore - Check if eslint.config.* exists → ensure the config's `ignores` entries cover required patterns - Check if .prettierrc* exists → create/verify .prettierignore - Check if .npmrc or package.json exists → create/verify .npmignore (if publishing) - Check if terraform files (*.tf) exist → create/verify .terraformignore - Check if .helmignore needed (helm charts present) → create/verify .helmignore If ignore file already exists: Verify it contains essential patterns, append missing critical patterns only If ignore file missing: Create with full pattern set for detected technology Common Patterns by Technology (from plan.md tech stack): - Node.js/JavaScript/TypeScript: `node_modules/`, `dist/`, `build/`, `*.log`, `.env` - Python: `__pycache__`, `*.pyc`, `.venv/`, `venv/`, `dist/`, `*.egg-info/` - Java: `target/`, `*.class`, `*.jar`, `.gradle/`, `build/` - C#/.NET: `bin/`, `obj/`, `*.user`, `*.suo`, `packages/` - Go: `*.exe`, `*.test`, `vendor/`, `*.out` - Ruby: `bundle/`, `log/`, `tmp/`, `*.gem`, `vendor/bundle/` - PHP: `vendor/`,`

`*.log`, `*.cache`, `*.env` - **Rust**: `target/`, `debug/`, `release/`, `*.rs.bk`, `*.rlib`, `*.prof`,
`.idea/`, `*.log`, `*.env` - **Kotlin**: `build/`, `out/`, `gradle/`, `*.idea/`, `*.class`, `*.jar`, `*.iml`,
.log`, `.env` - **C++**: `build/`, `bin/`, `obj/`, `out/`, `*.o`, `*.so`, `*.a`, `*.exe`, `*.dll`, `*.idea/`,
.log`, `.env` - **C**: `build/`, `bin/`, `obj/`, `out/`, `*.o`, `*.a`, `*.so`, `*.exe`, `*.dll`,
`autom4te.cache/`, `config.status`, `config.log`, `*.idea/`, `*.log`, `*.env` - **Swift**: `build/`,
`DerivedData/`, `*.swiftpm/`, `Packages/` - **R**: `Rproj.user/`, `Rhistory`, `RData`,
`.Ruserdata`, `*.Rproj`, `packrat/`, `renv/` - **Universal**: `DS_Store`, `Thumbs.db`, `*.tmp`,
.swp`, `.vscode/`, `*.idea/` **Tool-Specific Patterns**: - **Docker**: `node_modules/`, `*.git/`,
`Dockerfile`, `*.dockerignore`, `*.log`, `*.env`, `coverage/` - **ESLint**: `node_modules/`,
`dist/`, `build/`, `coverage/`, `*.min.js` - **Prettier**: `node_modules/`, `dist/`, `build/`,
`coverage/`, `package-lock.json`, `yarn.lock`, `pnpm-lock.yaml` - **Terraform**: `*.terraform/`,
.tfstate`, `.tfvars`, `*.terraform.lock.hcl` - **Kubernetes/k8s**: `*.secret.yaml`, `secrets/`,
`.kubel/`, `kubecfg/`, `*.key`, `*.crt` 5. Parse tasks.md structure and extract: - **Task
phases**: Setup, Tests, Core, Integration, Polish - **Task dependencies**: Sequential vs
parallel execution rules - **Task details**: ID, description, file paths, parallel markers [P] -
Execution flow: Order and dependency requirements 6. Execute implementation following
the task plan: - **Phase-by-phase execution**: Complete each phase before moving to the
next - **Respect dependencies**: Run sequential tasks in order, parallel tasks [P] can run
together - **Follow TDD approach**: Execute test tasks before their corresponding
implementation tasks - **File-based coordination**: Tasks affecting the same files must run
sequentially - **Validation checkpoints**: Verify each phase completion before proceeding 7.
Implementation execution rules: - **Setup first**: Initialize project structure, dependencies,
configuration - **Tests before code**: If you need to write tests for contracts, entities, and
integration scenarios - **Core development**: Implement models, services, CLI commands,
endpoints - **Integration work**: Database connections, middleware, logging, external
services - **Polish and validation**: Unit tests, performance optimization, documentation 8.
Progress tracking and error handling: - Report progress after each completed task - Halt
execution if any non-parallel task fails - For parallel tasks [P], continue with successful tasks,
report failed ones - Provide clear error messages with context for debugging - Suggest next
steps if implementation cannot proceed - IMPORTANT For completed tasks, make sure to
mark the task off as [X] in the tasks file. 9. Completion validation: - Verify all required tasks are
completed - Check that implemented features match the original specification - Validate that
tests pass and coverage meets requirements - Confirm the implementation follows the
technical plan Note: This command assumes a complete task breakdown exists in tasks.md. If
tasks are incomplete or missing, suggest running `/speckit.tasks` first to regenerate the task
list.`

Mandatory Post-Execution Hooks

****You MUST complete this section before reporting completion to the user.**** Check if
``.specify/extensions.yml`` exists in the project root. - If it does not exist, or no hooks are
registered under ``.hooks.after_implement``, skip to the Completion Report. - If it exists, read it
and look for entries under the ``.hooks.after_implement`` key. - If the YAML cannot be parsed or
is invalid, skip hook checking silently and continue to the Completion Report. - Filter out hooks
where ``.enabled`` is explicitly ``.false``. Treat hooks without an ``.enabled`` field as enabled by

default. - For each remaining hook, do **not** attempt to interpret or evaluate hook ``condition`` expressions: - If the hook has no ``condition`` field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty ``condition``, skip the hook and leave condition evaluation to the HookExecutor implementation - For each executable hook, output the following based on its ``optional`` flag: - **Mandatory hook** (``optional: false``) — **You MUST** emit ``EXECUTE_COMMAND:`` for each mandatory hook: ```` ## Extension Hooks`
Automatic Hook: {extension} Executing: ``/{command}`` ``EXECUTE_COMMAND:`
`{command}`` ````` After emitting the block above you **MUST** actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal ``{command}`` id shown above, e.g. a skills-mode agent runs it as ``/skill:speckit-...`` or ``$speckit-...``). Emitting the block alone does not run the hook. - **Optional hook** (``optional: true``): ```` ## Extension Hooks` **Optional Hook**: {extension} Command: ``/{command}`` Description: {description} Prompt: {prompt} To execute: ``/{command}`` `````

Completion Report

Report final status with summary of completed work.

Done When

- [] All tasks in tasks.md completed and marked ``[X]`` - [] Implementation validated against specification, plan, and test coverage - [] Extension hooks dispatched or skipped according to the rules in Mandatory Post-Execution Hooks above - [] Completion reported to user with summary of completed work

source: `.github/agents/speckit.implement.agent.md` | commit: n/a